Rightsize: team brief and build plan

Oct 4, 2026 · @Roni

Rightsize is a copilot inside a bank's business app. It warns a small-business owner weeks before cash runs short, shows how big the gap really is, and lays out the cheapest ways to close it, including not borrowing. We submit to the FinTechathon by Oct 20, 2026, 23:59 Beijing time (15:59 UTC).

This version replaces the 1 Oct brief. Where the two differ, this one counts. On 4 Oct the chat screen and the payment forecast are built, on two separate branches, and nothing connects them yet.

What changed since 1 Oct

Topic	1 Oct brief	Now
AI model	Claude, a paid service, through its API	Qwen 2.5 (7B), a free model on our own computer, through Ollama
Forecast	Simulate the next 60 days thousands of times	A trained model gives every unpaid invoice a chance of payment for each coming day
History	12 months	Almost five years, January 2022 to September 2026
Demo company	About 40 customers, 12 staff, 30 days to pay, busy in summer and December	500 customers, 15, 30 or 60 days to pay, even sales through the year, staff not set
Screen	Not specified	A web chat (React) with a small Python server (FastAPI)
Order of work	Bank and actions first, screen last	Screen and forecast came first; bank, actions and permission code are next

The three jobs, the advise-only rule, the permission table, the six owner questions and the dates are unchanged.

What we are building

Rightsize is our entry for the 2026 Shenzhen FinTechathon, International AI Track, Topic E (SME Finance Copilot). It has three jobs:

1. **Watch.** It forecasts the company's cash every day and speaks first, weeks ahead, when salaries or the tax payment are at risk.
2. **Size.** It shows the gap in three scenarios: likely, cautious and worst case.

3. **Close.** It lists every option with its total cost, including "wait" and "remind the late client", marks one as recommended, and waits for the owner's decision.

The agent advises and the owner acts. It never borrows, repays or pays anyone.

Every number the agent shows comes from ordinary Python code. The AI model reads the owner's message, decides which calculation to run, and explains the result. It does no arithmetic of its own.

Demo company. A Tbilisi food-and-drink wholesaler selling to 500 businesses across Georgia: restaurants, cafés, hotels, bars, caterers and grocery stores. About half are in Tbilisi.

- It invoices about GEL 13 million a year, in about 12,000 invoices. The typical invoice is about GEL 700.
- Customers get 15, 30 or 60 days to pay. 57% of paid invoices were paid late. The typical invoice is paid two days after its due date, and one in 25 is more than 30 days late.
- The books stop on 3 Oct 2026. On that day customers owe GEL 1.48 million, of which GEL 332,000 is overdue, spread over 268 invoices and 157 customers.
- Customers pay more slowly when the due date falls on their own payroll days (12th to 15th), at month end, and around weekends and Georgian public holidays.
- Sales are even through the year. The books have no busy season.
- The books hold invoices only. The company's bank balance, staff, salaries, taxes and stock purchases are not in the data yet. Our working dates stay: salaries on the 10th, taxes on the 15th.
- Amounts are in lari. The bank is fictional. Every record is generated by code, and none comes from a real company.

These figures come from the generated books, re-run on 4 Oct.

What is built

Part	What it does	State on 4 Oct
Chat screen	A web page where the owner types and replies appear word by word. Conversations are saved in the browser, up to 20 messages each.	Built, on `feat/basic-chat`
Server	A small Python program that passes messages between the screen and the AI model.	Built, on `feat/basic-chat`
AI model	Qwen 2.5 (7B), a free model that runs on our own computer through Ollama. Nothing is sent to an outside service.	Connected. It has no access to the company's data and is told not to invent figures.

Part	What it does	State on 4 Oct
Invoice history	Code that generates the demo company's books: 50,319 invoices with issue, due and payment dates.	Built, on `feat/payment-forecast`
Payment forecast	For every unpaid invoice, the chance it is paid within 7, 14, 30 and 60 days.	First model built, on `feat/payment-forecast`. Calibration and the final test are open.
Company's own money	Bank balance, salaries, taxes and stock purchases, plus the bank's products.	Not started
Actions	The look-ups and calculations the AI model may ask for.	Not started
Cash gap	Money expected in minus money due out, as likely, cautious and worst case.	Not started
Permission code and log	Enforces the table in "What the agent may do" and records every action.	Not started
Test scripts	Run the owner questions and trick attempts and report the pass rate.	Not started

How the forecast works. The model looks at one unpaid invoice on one day and gives the chance it is paid that day. Chaining the days gives the chance it is paid within a week, two weeks, a month or two months. It learns from invoices issued in 2022 to 2024, its settings are chosen on 2025, and 2026 is kept untouched for the final test. A simple forecast built from past averages sits beside it, and the model has to beat it.

The training and tuning code has been run: later scripts name the winning setting. The scores are not in the repository, since generated files are not committed. The data is generated, so good scores will show that the method works on our own assumptions, not that it predicts real Georgian customers.

What the owner asks

These six questions start our test list. Each one gets a correct answer worked out by hand.

1. "Can I pay salaries on the 10th and taxes on the 15th?"
2. "Who owes me money, and who is late?"
3. "Send reminders to the late ones."
4. "I'm short. What are my options?"
5. "Client A paid. Do I still need the overdraft?"
6. "Can I afford a GEL 20,000 stock order next month?"

Question 2 needs only the invoice data we already have, so it is the one to get working on screen first.

One exchange, with placeholder numbers:

> **Rightsize:** Heads-up. After salaries on the 10th and taxes on the 15th, you are likely to be GEL 8,000 short for about three weeks.
>
> **Owner:** How bad can it get, and what can I do?
>
> **Rightsize:** Likely case GEL 8,000. Cautious case GEL 14,000: nine in ten scenarios stay under this. Worst case GEL 30,000, if nobody pays. I recommend the reminder now and the overdraft as a fallback. Which do you want?

Option	Cost	Note
Remind Client A, who owes GEL 15,000	Free	Recommended first step
Overdraft, GEL 15,000 limit	About GEL 74 if you use GEL 8,000 for three weeks	Fallback
12-month loan of GEL 30,000	About GEL 2,800 in interest and fees	Covers the worst case and costs the most

> **Owner:** Remind them, and get the overdraft request ready.
>
> **Rightsize:** Here is the reminder text. Shall I send it? The overdraft request for GEL 15,000 is filled in and waiting in your bank app for you to submit.

The costs use made-up terms: an overdraft at 16% a year, and a loan at 15% a year plus a 1% fee. The finance owners replace them with real terms.

The amounts were written on 1 Oct for a smaller company. The company in the data invoices about GEL 1.1 million a month, so the finance owners also scale up the amounts here and in question 6.

What the agent may do

The organisers ask for three permission levels. Ours are enforced by code that sits between the agent and its actions, not by instructions to the AI model.

Action	Who does it	Level
Look up balances and invoices, forecast, compare options	Agent, without asking	Green
Set an alert, save a plan	Agent, after the owner says yes	Yellow
Send a payment reminder to a late client	Agent, after the owner approves the text	Yellow
Fill in an overdraft or loan request	Agent prepares it; the owner submits it in the bank app	Red: owner only
Borrow, repay, pay anyone	Owner, in the bank app	Red: owner only

The same code records every action in a log and locks the agent after repeated suspicious requests. The log is the evidence we submit.

None of this is built yet. Today the AI model can only talk.

How we are scored

The three criteria come from the official site. Ten teams reach the final.

Criterion	What judges look at	What we build for it
Task completion, 40%	Scripted tasks completed in a simulated environment	About 20 owner questions, each with a hand-worked correct answer. A script runs them all and reports the pass rate.
Security and compliance, 30%	Resisting four attacks: being talked into a transfer, leaking data, false claims of authority, and hidden instructions in documents. Serious failures cost heavily.	About 20 trick attempts across those four kinds, run by script. The agent cannot move money at all. It always shows the total cost and yearly rate, and says it is the bank's assistant.
Innovation and interaction, 30%	Novelty, real-world value, few messages to a result, asking when unclear, accepting corrections, declining out-of-scope requests	The agent speaks first, shows three scenarios and may advise against borrowing. One real business owner tries it.

Because the agent only advises, the 40% depends on its answers being right. The answer key is our most important document.

Who owns what

Two pieces are built, both by the same teammate.

Built	Where	By
Chat screen, server and the local AI model	`feat/basic-chat`	nianordv
Invoice history and payment forecast	`feat/payment-forecast`	nianordv, starting from a teammate's first generator

The rest keeps the 1 Oct proposal, minus those two pieces. The screen was in the Tech 1 row and the forecast in the fifth member's row. Replace the role labels with names and move rows as you see fit.

Who	Still to build	Writes at the end
Tech 1 (backend, data)	The pretend bank and the company's own money, the agent's actions, the permission code and log	README and execution evidence
Tech 2 (ML)	The agent's conversation, where the local model chooses actions, and the scripts that run the owner questions and trick attempts	Technical document
Fifth member (astrophysics)	The cash gap in three scenarios, built on the payment forecast, then the option costs and the what-if question	Evaluation part of the technical document
Finance 1	Real product terms and the demo company's numbers, sized for a company that invoices GEL 13 million a year	Deck, and the story in the video
Finance 2	The answer key and the list of trick attempts	Security self-assessment

The cash-gap row assumes the astrophysics member codes in Python. If not, whoever built the forecast takes the cash gap and they take the documents.

AI coding assistants write much of the code with the tech owners. Each owner must still be able to explain their part at the defence in Shenzhen.

Build plan

The dates and checkpoints are the ones set on 1 Oct. The cells show where each piece stands on 4 Oct.

	Set up **2 to 3 Oct**	**First question** **4 to 8 Oct**	**All questions** **9 to 14 Oct**	**Documents** **15 to 20 Oct**
Chat and agent	**Done** repository, chat screen, server, local model	Merge both branches Agree the action list Model calls its first action	All 20 questions Test scripts 20 trick attempts	Technical document
Bank, actions, permissions	**Open** data tables agreed	Books in a database: invoices, balance, salaries, taxes Look-up actions	Permission code, log Reminders, alerts	README Execution evidence
Forecast	**Done** method chosen, invoice history, first model	Forecast offered as an action Cash gap, version 1	Option costs What-if question Calibration and final test	Evaluation results and charts
Finance, two people	Product terms Company numbers	First 10 questions with answers	Full answer key Trick-attempt list Real owner tries it	Deck and video Security self-assessment
Checkpoint, done when	**3 Oct** Reached for the repository, chat and invoice data **Open** action list, company numbers, registration to confirm	**8 Oct** First question works on screen	**14 Oct** All questions pass No new features	**18 Oct** Full package sent **20 Oct** Final deadline

Each column is a phase, and its bottom cell says what must be true before the next phase starts. From 15 Oct nobody adds features, and 19 and 20 Oct are spare days. Finance work is not kept in the repository, so its row shows the plan, not the state.

We are ahead on the screen and the forecast, and behind on the bank, the actions and the permission code. The 8 Oct checkpoint depends on those three.

If a checkpoint slips, cut in this order: the what-if question, the alerts, then the extra test companies.

What we are leaving out

Each item below is out of scope, to fit the 16 days left.

- The agent moving money or taking a loan.
- Reading invoice PDFs or photos. One pasted-text path stays, to show the agent ignoring a hidden instruction inside an invoice.
- Fifty synthetic companies. We have one company with 500 customers. Two or three variants for tests come only if time allows.
- Part payments, refunds and money recovered after a write-off. In the data an invoice is paid once, in full, or not at all.
- Factoring, inventory sales, early-payment discounts and multiple currencies.
- Real bank connections, a mobile app and user accounts.

Next steps, 4 to 8 Oct

Registration and the organisers' answers still come first, because both can take days.

- [ ] Register by email: the team form, plus each member's ID document and proof of enrolment. All five must be full-time students at universities outside China. The site lists fintechathon@ican-x.com for this track; the rules list wangyr@pbcsf.tsinghua.edu.cn.
- [ ] Ask the organisers three questions in the same email:
    1. Are the scripted tasks supplied by you, or defined by each team?
    2. Does the 0 to 5 point bonus cover demonstrated real-world deployment, or only on-chain work?
    3. Is the demo video required?
- [x] Tech: create the GitHub repository.
- [ ] Tech: merge `feat/basic-chat` and `feat/payment-forecast` into `dev`. They merge without conflicts.
- [ ] Tech: each of us installs Ollama, downloads the model with `ollama pull qwen2.5:7b` (about 5 GB) and runs the chat.
- [ ] Tech: add `lightgbm` to `ml/requirements.txt`, and update `ml/README.md`, which still says the model is not trained.
- [ ] Tech: add the company's own money to the books: opening balance, salaries, taxes and stock purchases.
- [ ] Tech: agree the action list, then make question 2 work on screen.
- [ ] Finance: write the terms for an overdraft or credit line and for a term loan (rate, fees, minimum term), and confirm the salary and tax dates.

- [ ] Finance: set the company's numbers for its real size: staff, monthly salary bill, taxes, stock purchases and opening balance.
- [ ] Finance: check the demo company and the six owner questions, with the real owner if possible.
- [ ] Everyone: read this brief and correct anything that is wrong.

Sources

- Official site: judging criteria, schedule and registration process, read on 1 Oct 2026.
- The organisers' rules PDF, held by the team lead: topics, deadline and submission checklist.
- Our repository: branches `feat/basic-chat` and `feat/payment-forecast`, read on 4 Oct 2026.
- `docs/data-generation-and-handling.md` and `ml/README.md` on `feat/payment-forecast`: how the invoice history is generated and prepared.
- Submission templates served by the official site. They are not linked from its visible pages, so treat them as drafts until the organisers confirm.
  - Technical document
  - Security self-assessment
  - Execution evidence
  - Demo video guide
  - README template
  - Presentation deck
  - Team information form